

# Deterministic Agentic Swarm Control: Commitment Boundaries for High-Stakes Multi-Agent AI

Anteneh Tessema  
*AI Engineer*  
*Yaya Systems*

May 2026

## Abstract

Agentic AI systems increasingly coordinate across long workflows, invoke tools, and write to shared memory. In high-stakes domains, this creates a reliability hazard: probabilistic cognition can propagate through agent dependencies and become durable, consequential system state.

This paper introduces Deterministic Agentic Swarm Control (DASC), a commitment-control architecture for Service-as-a-Software agentic systems. DASC separates probabilistic agent cognition from deterministic commitment by routing consequence-bearing actions through a boundary kernel that evaluates typed intents against policy, provenance, state versioning, authority, and validation constraints.

The paper formalizes the multi-agent state model and introduces implementation mechanisms for common deployment hazards, including TOCTOU contention, epistemic contamination, non-idempotent external actions, tainted memory, and agent spoofing. We discuss how optimistic concurrency control, information-flow labels, attestation, provenance chains, and optional proof-carrying ledger entries can strengthen DASC implementations. DASC does not make agent cognition deterministic, nor does it guarantee semantic correctness. Its narrower contribution is a systems architecture for reducing the probability that cognitive faults become committed state.

**Keywords:** agentic AI; multi-agent systems; deterministic commitment; runtime assurance; typed intents; provenance; information flow control; optimistic concurrency control; bitemporal ledger; Service-as-a-Software

## 1 Introduction

As artificial intelligence transitions toward autonomous agentic operation, systems are increasingly designed to maintain state, invoke tools, retrieve documents, and coordinate asynchronously. This paradigm shift alters the fundamental safety boundary of AI. A hallucination in a single-turn chatbot is an inconvenience; a hallucination written into a shared memory ledger, translated into production code, or used as the basis for financial action becomes a material system failure.

In this paper, Service-as-a-Software (SaaS) refers to software systems in which persistent business processes are executed by stateful agentic services rather than by fixed application logic alone. Existing guardrails, retrieval systems, and orchestration frameworks improve agent behavior within these paradigms but often leave the commitment boundary implicit. They may constrain outputs, route tasks, or retrieve context, yet still allow unsupported intermediate results to be written into memory, used as downstream evidence, or translated into external action.

DASC treats that transition—from uncertain cognition to authoritative state—as the primary object of control. DASC does not make agent cognition deterministic; it makes consequence-bearing commitment deterministic, replayable, evidence-bound, and policy-governed.

## 2 Threat Model and Design Goals

### 2.1 Design Goals

DASC aims to:

1. prevent unsupported cognition from becoming committed state;
2. make high-risk commitments replayable;
3. preserve provenance and evidence integrity;
4. prevent low-integrity data from silently influencing high-risk actions;
5. handle asynchronous state contention;
6. govern non-idempotent actions through compensation;
7. support high-assurance implementations without requiring every deployment to use cryptographic proofs.

### 2.2 Non-Goals

DASC does not:

1. eliminate hallucinations;
2. prove semantic truth;
3. replace secure tool design;
4. guarantee evidence correctness;
5. require zk-SNARKs or TEEs for every deployment;
6. make human review infallible.

## 3 Related Work

### 3.1 LLM Orchestration Frameworks

Frameworks such as AutoGen [1] and ReAct [2] excel at managing multi-agent dialogue and graph-based state routing. However, these frameworks generally lack an explicit commitment boundary, often treating the outputs of probabilistic consensus as authoritative.

### 3.2 Runtime Assurance

Traditional runtime assurance supervises untrusted controllers with a trusted safety envelope [3]. DASC extends this to epistemic digital state, preventing probabilistic text generation from violating policy.

### 3.3 Information Flow Control and Taint Tracking

DASC borrows from classic Information Flow Control (IFC) [4] but applies integrity taint labels to agent memory and typed intents to track context poisoning across multi-turn swarms.

### 3.4 Provenance and Transaction Processing

DASC adapts database transaction protocols (Optimistic Concurrency Control) and distributed system recovery mechanisms (Sagas) to asynchronous agent swarms. Furthermore, it leverages content-addressed storage and provenance standards (C2PA) [6] to secure agent retrieval pipelines.

### 3.5 Proof-Carrying Computation

While DASC allows for Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs) [5], these are treated as optional compression mechanisms for bitemporal ledgers, not as the source of semantic truth.

## 4 Formal Problem Definition

We formalize the multi-agent orchestration environment as a tuple  $\mathcal{M} = \langle \mathcal{A}, \mathcal{S}, \mathcal{I}, \kappa \rangle$ , where:

- $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$  is a set of autonomous probabilistic agents.
- $\mathcal{S}$  represents the authoritative global state.
- $\mathcal{I}$  is the set of Typed Intents proposed by agents.
- $\kappa$  is the Deterministic Control Kernel.

A commitment does not merely add an intent to a ledger; it executes a transition. The state transition is defined as:

$$S_{t+1} = \begin{cases} \delta(S_t, I) & \text{if } \kappa(I, S_t, P_t, E_t, L_t) = \text{COMMIT} \\ S_t & \text{otherwise} \end{cases} \quad (1)$$

Where  $\delta$  is the deterministic transition function,  $P_t$  is policy state,  $E_t$  is evidence state, and  $L_t$  is ledger state. The kernel maps to a multi-outcome decision space:

$$\kappa : \mathcal{I} \times \mathcal{S} \times \mathcal{P} \times \mathcal{E} \times \mathcal{L} \rightarrow \{\text{COMMIT}, \text{REJECT}, \text{REQUEST\_EVIDENCE}, \text{ESCALATE}\} \quad (2)$$

Our primary safety metric is minimizing  $P(B|F_c)$ , the conditional probability of a boundary breach ( $B$ ) given a cognitive fault ( $F_c$ ).

## 5 Core DASC Architecture

### 5.1 Bounded State-Space Verification

Bounded Agentic Sandboxes do not restrict internal reasoning. They restrict commitment egress. Agents may plan freely inside the sandbox, but any attempt to modify authoritative state must exit as a typed intent. The kernel does not evaluate the subjective quality of the agent’s reasoning. It evaluates only deterministic predicates: schema validity, action admissibility, evidence provenance, state version compatibility, authority, risk tier, taint label, and compensation requirements.

## 5.2 Concurrency, Non-Idempotence, and State Safety

Asynchronous swarms create Time-of-Check to Time-of-Use (TOCTOU) vulnerabilities. DASC requires Optimistic Concurrency Control (OCC) via version vectors ( $V_s$ ).

For replayability, if an intent is rejected due to a version conflict, any backoff schedule must be derived from deterministic inputs such as `intent_id`, `actor_id`, and conflict class. Non-overlap is defined at the smallest state unit governed by the target registry (e.g., file path and line range for code).

Treating reversibility as a simple state rollback is a fallacy for external actions. DASC incorporates Sagas. The kernel should reject non-idempotent external actions unless the compensating intent is itself schema-valid, authorized, policy-compliant, and executable. If compensation fails, DASC places the affected workflow or commitment class into a halted state pending human escalation.

## 6 Information Flow, Provenance, and Execution Security

### 6.1 IFC and Integrity Taint Tracking

DASC primarily uses integrity labels. A low-integrity source should not be allowed to justify a high-integrity commitment without validation or declassification. Declassification requires deterministic validation, evidence provenance, and, for high-risk tiers, human authority. Taint propagation is modeled as  $\tau(I) = \tau(m) \sqcup \tau(a_i)$ .

### 6.2 Instruction/Data Separation and Provenance

DASC uses instruction/data separation rather than relying on agents to ignore malicious text. Retrieved evidence is passed to validators as inert data, not executable instruction. Where available, cryptographic provenance chains (e.g., C2PA signatures) strengthen authenticity. Absence of such provenance need not imply automatic rejection; instead, the kernel may lower evidence trust, require corroboration, or escalate according to risk tier.

## 7 Optional Cryptographic Proof-Carrying Ledger

DASC can optionally use proof-carrying ledger entries for high-volume swarms. Rather than storing every intermediate validation trace inline, the system stores content-addressed artifacts and records a zk-SNARK proof that the deterministic kernel was executed over committed inputs. These proofs improve audit compression. They do not prove that the underlying evidence is true or that the policy is correct; they prove only that the specified validation computation was performed against committed inputs under a specified circuit.

## 8 The DASC Protocol (RFC)

To establish interoperability, DASC requires a formalized protocol. Implementations are categorized by assurance levels.

### 8.1 Core DASC (Required)

A DASC-compliant intent **MUST** include: `intent_id`, `actor_agent`, `action_type`, `target_artifact`, `risk_tier`, `evidence_references`, `requested_transition`, and `authorization_context`. The kernel **MUST** evaluate policy checks and maintain an audit ledger.

## 8.2 Enterprise DASC (Recommended)

A Tier 3 or higher intent **SHOULD** include: `state_version_vector` (OCC), `evidence_provenance`, `validation_artifacts`, `human_authority_requirement`, and `taint_labels`. A non-idempotent external action **MUST** include a `compensating_intent` and a `dead_letter_escalation` route.

## 8.3 High-Assurance DASC (Optional)

For critical infrastructure, deployments **MAY** require Trusted Execution Environments (TEEs) for agent attestation, formal contracts, and proof-carrying ledger entries (zk-SNARKs).

# 9 Evaluation Methodology and Limitations

## 9.1 Residual Risk Analysis

DASC mitigates primary swarm attack vectors architecturally, though residual risks remain:

| Threat               | DASC Control                | Residual Risk                                      |
|----------------------|-----------------------------|----------------------------------------------------|
| TOCTOU Contention    | OCC version vectors         | Semantic conflicts may evade field-level detection |
| Privilege Escalation | IFC taint propagation       | Declassification policy may be flawed              |
| Agent Spoofing       | Attestation signatures      | TEE trust and key management assumptions           |
| Prompt Injection     | Instruction/data separation | Agents may still summarize malicious content       |
| Evidence Poisoning   | Provenance chains           | Signed evidence can still be false                 |
| Non-idempotent Fails | Sagas and compensation      | Compensation may fail or be incomplete             |
| Policy Misconfig     | Policy versioning / replay  | Deterministic wrong decisions                      |
| Approval Failure     | Structured reason codes     | Human rubber-stamping                              |

Table 1: DASC Threat and Residual Risk Matrix

## 9.2 Computational Overhead Bounds

The verification boundary introduces latency. OCC checks are typically constant or logarithmic relative to indexed state objects. Attestation depends on certificate-chain verification. Proof generation for a zk-ledger depends on the chosen proof system and circuit size. DASC treats cryptographic proof generation as an optional high-assurance write-path optimization, not as a mandatory requirement.

## 9.3 Limitations

DASC does not guarantee that committed actions are semantically correct. It guarantees only that selected commitment predicates are evaluated deterministically over recorded inputs. Cryptographic provenance proves origin and integrity, not truth. TEEs introduce hardware trust assumptions. zk proofs prove computation under a circuit, not policy correctness. IFC taint labels depend on correct source classification. OCC prevents stale writes but does not resolve semantic conflicts. Human escalation remains vulnerable to approval fatigue.

# 10 Conclusion

By standardizing typed intents and deterministic commitment boundaries, DASC helps shift high-stakes agentic systems from probabilistic consensus toward auditable, policy-governed commitment. Separating core architecture from optional high-assurance cryptography provides a

deployable, defensible framework for reducing the probability that cognitive faults become authoritative state.

## A DASC-1.0: Typed Intent Specification

The formalized schema demonstrating Core and Enterprise DASC requirements.

```
1 {
2   "intent_id": "INT-2026-00041",
3   "actor_agent": "migration_agent_07",
4   "action_type": "commit_code_translation",
5   "target_artifact": "payments/batch_settlement.py",
6   "risk_tier": 3,
7   "state_version_vector": {
8     "mainframe_repo_etag": "W/\\"4d9f0b7e2\\"",
9     "target_repo_etag": "W/\\"8a2c1f9b0\\""
10  },
11  "ifc_taint_label": "EXTERNAL_UNVERIFIED_READ",
12  "policy_version": "migration-policy-v3.2.1",
13  "requested_transition": {
14    "operation": "apply_patch",
15    "patch_hash": "sha256:4d9f0b7e2..."
16  },
17  "preconditions": [
18    "source_lines_mapped",
19    "regression_tests_passed"
20  ],
21  "postconditions": [
22    "translation_recorded",
23    "rollback_point_created"
24  ],
25  "evidence": [
26    {
27      "evidence_id": "EVD-2026-00091",
28      "source_type": "static_code_repository",
29      "span_type": "line_range",
30      "provenance_proof": {
31        "c2pa_signature": "sig-9x8f7a..."
32      },
33      "span": {
34        "start_line": 214,
35        "end_line": 247,
36        "content_hash": "sha256:8a2c1f9b0..."
37      }
38    }
39  ],
40  "validation_artifacts": [
41    {
42      "artifact_type": "regression_test_report",
43      "result": "passed",
44      "hash": "sha256:c8201fae..."
45    }
46  ],
47  "human_authority": {
48    "required": true,
49    "role": "senior_migration_reviewer"
50  },
51  "compensation_plan": {
52    "saga_type": "compensating_intent",
53    "revert_intent": "revert_translation_patch",
54    "fallback": "dead_letter_escalation_queue"
55  }
56 }
```

## B Example Rejected Commitment Decision

```
1 {
2   "intent_id": "INT-2026-00041",
3   "decision": "REJECT",
4   "reason_codes": [
5     "NO_ACTIVE_SOURCE_SPAN",
6     "REGRESSION_TEST_FAILURE"
7   ],
8   "failed_predicates": [
9     "EvidenceValid",
10    "TestsSatisfied"
11  ],
12  "evidence_conflicts": [
13    {
14      "evidence_id": "EVD-2026-00091",
15      "issue": "comment is obsolete and does not correspond to active logic"
16    }
17  ],
18  "ledger_entry": {
19    "decision_time": "2026-05-01T14:44:00Z",
20    "recorded_by": "deterministic_commitment_kernel",
21    "replay_available": true
22  }
23 }
```

## References

- [1] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*.
- [2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*.
- [3] Havelund, K., & Rosu, G. (2004). An overview of the runtime verification tool Java PathExplorer. *Formal Methods in System Design*, 24(2), 189-224.
- [4] Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308.
- [5] Ben-Sasson, E., Chiesa, A., Tromer, E., & Virza, M. (2014). Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture. In *23rd USENIX Security Symposium*, 781-796.
- [6] Coalition for Content Provenance and Authenticity (C2PA). (2024). *Technical Specification for Digital Provenance*.